PROGRAMMING 621 ASSIGNMENT

RICHFIELD

Name and Surname: Keshav Ramphal

Student ITS No: 402503605

Qualification: Diploma In Information Technology Year of Study: 2 Semester: 1

Assignment due date: 3 May 2026 Date submitted: 3 May 2026

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

✔ Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard
✔ Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

✔ Table of Contents Introduction
✔ Main Body (with relevant subheadings) Conclusion
✔ References
✔ Submit the assignment in PDF format on Moodle. Use the
✔ specified cover page provided.
✔ Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature:  Date: 03/05/2026

Contents

Introduction

What is programming and how is it used?

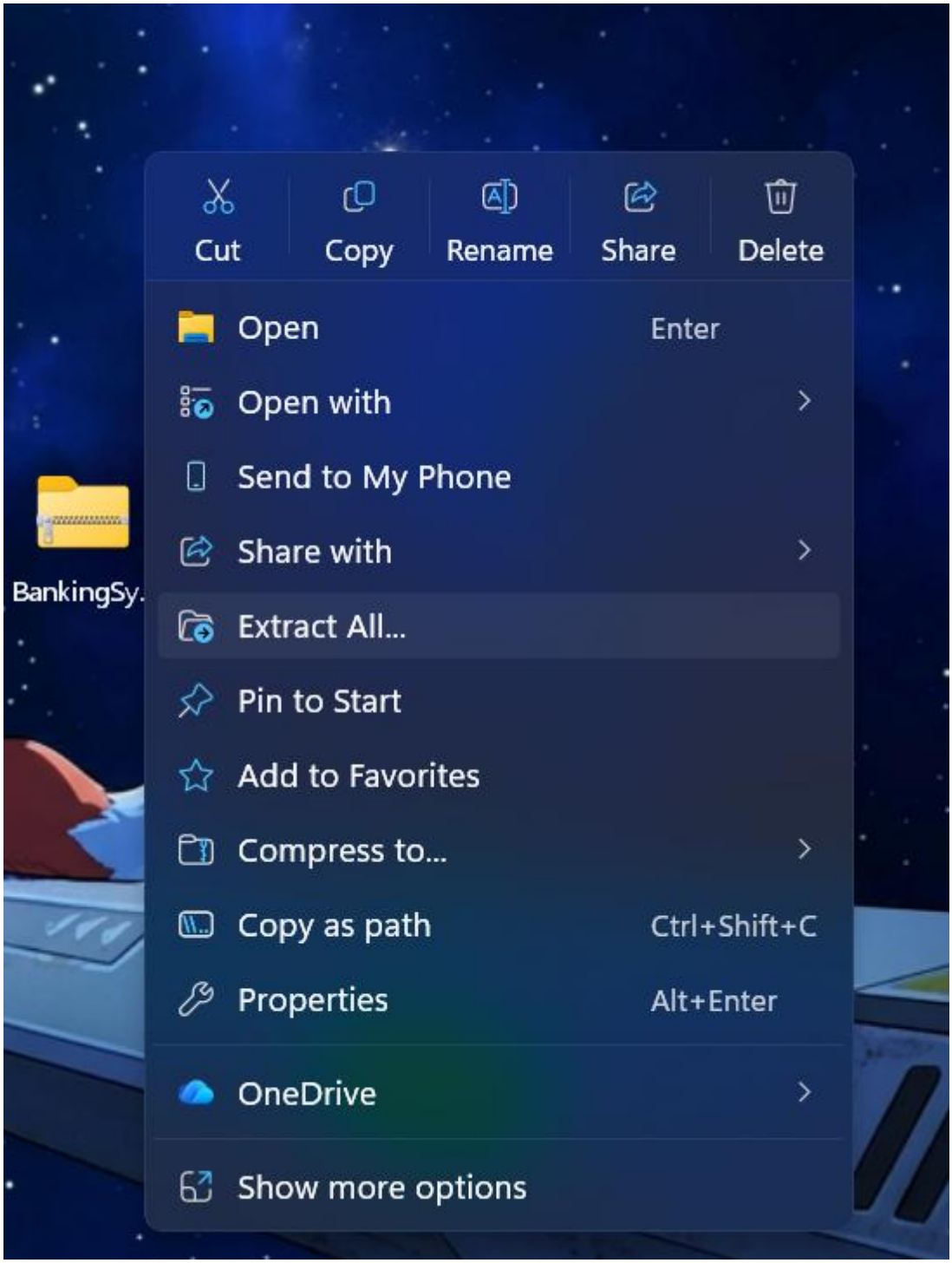
Is the development of set applications and is the back bine when developing anything regarding technology in a whole, and in modern day standards we have hundreds of languages in terms of programming for set aspects for example one language will be good at coding with mobile device while another consoles etc. (w3schools, 2026).

Main Part

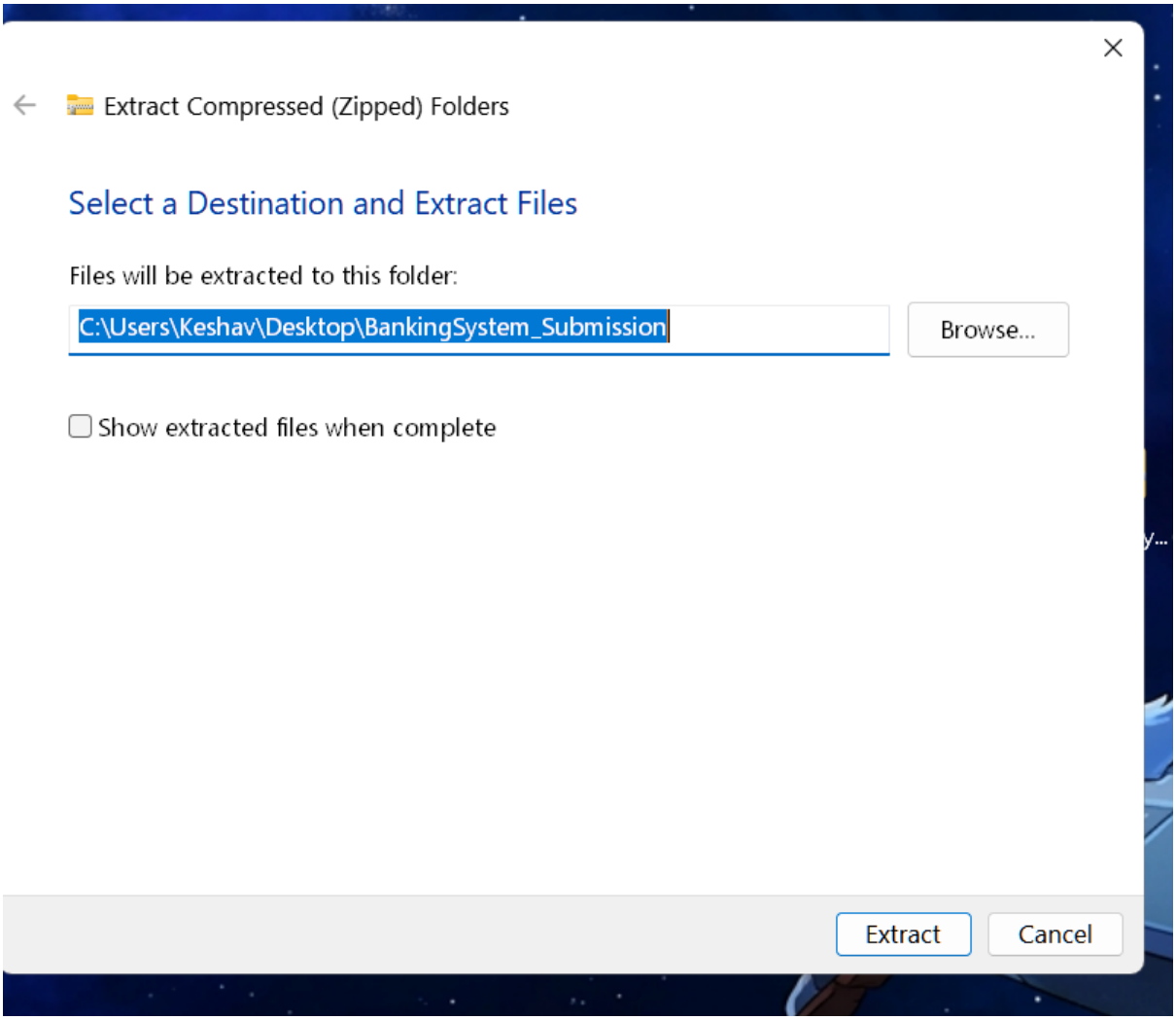
I will be presenting the proof of concept that the code works via screenshots and where and also all code is in this doc and in the original README.txtx

First I will present the screenshots and how to actually get the data to do stuff then the code

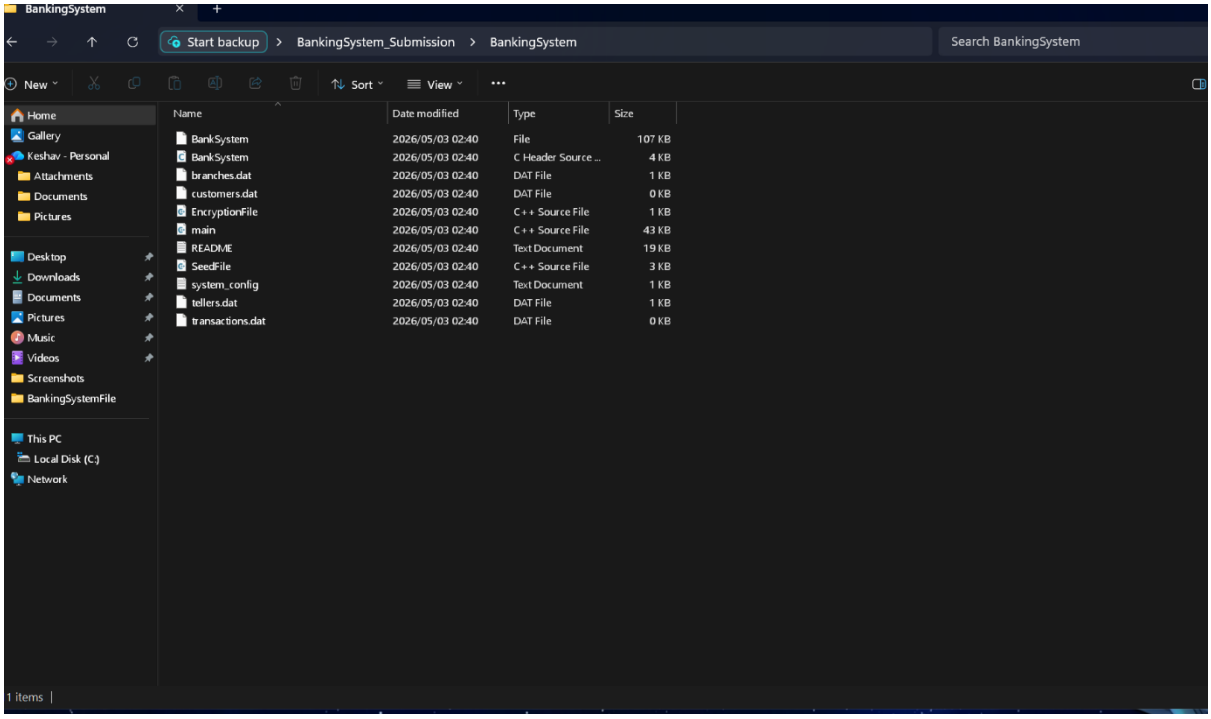
One is to extract the file

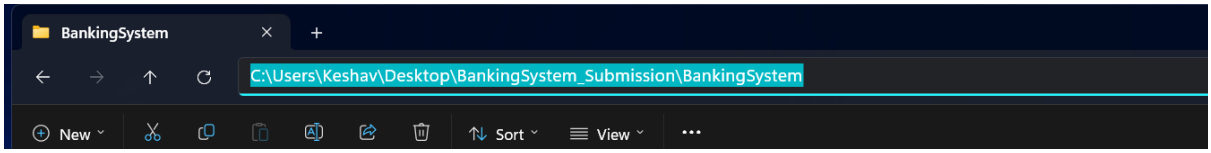


When you extract the file then save the path to desktop and you will get a file as seend in the below image

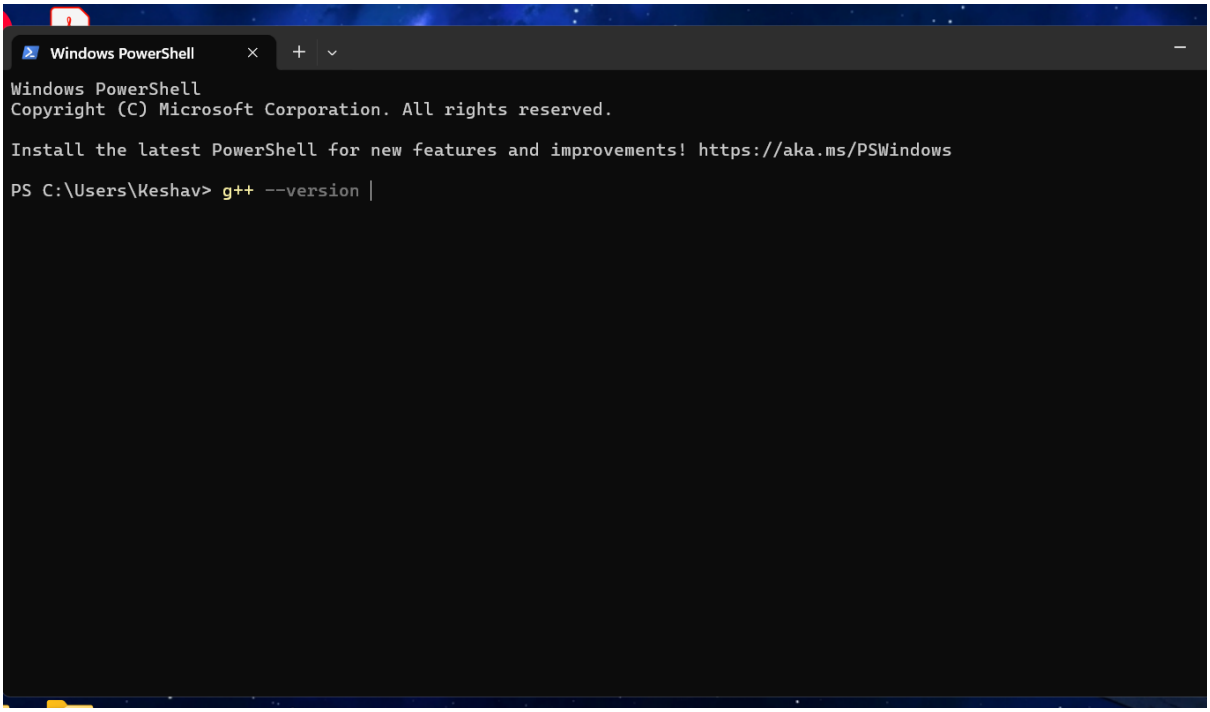


Go into the file and these are where all the siles are, there after copy the path as seen below

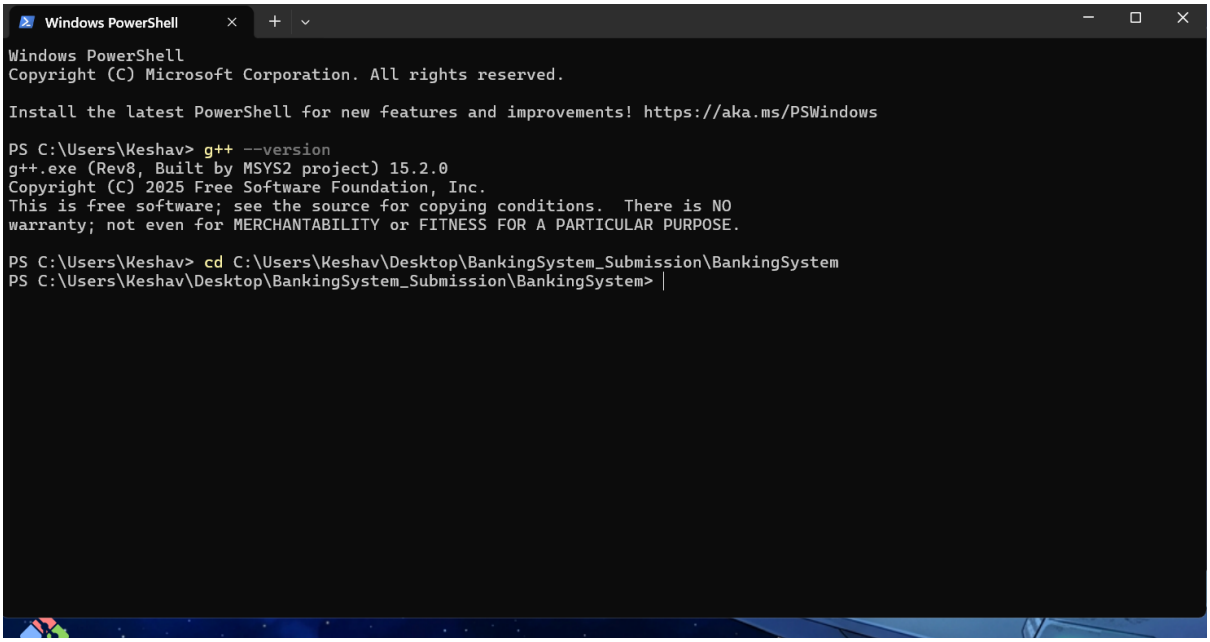




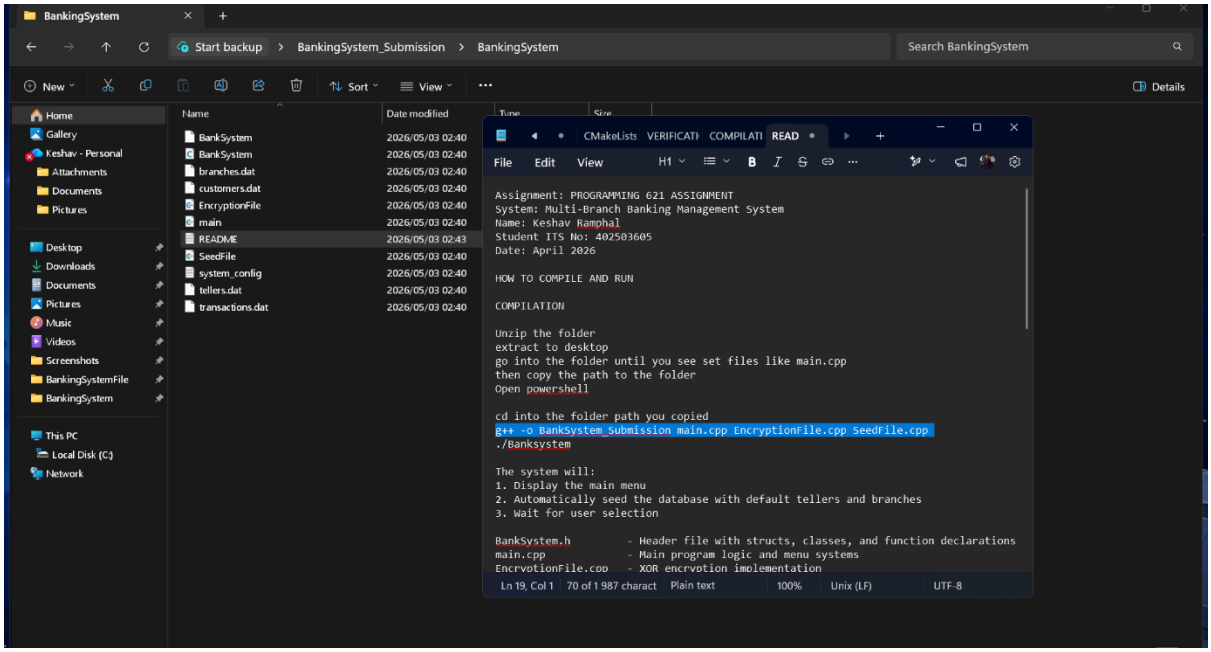
Next open Control panel and then go to powerpoint and check if you have C++ compiler installed, if not get it

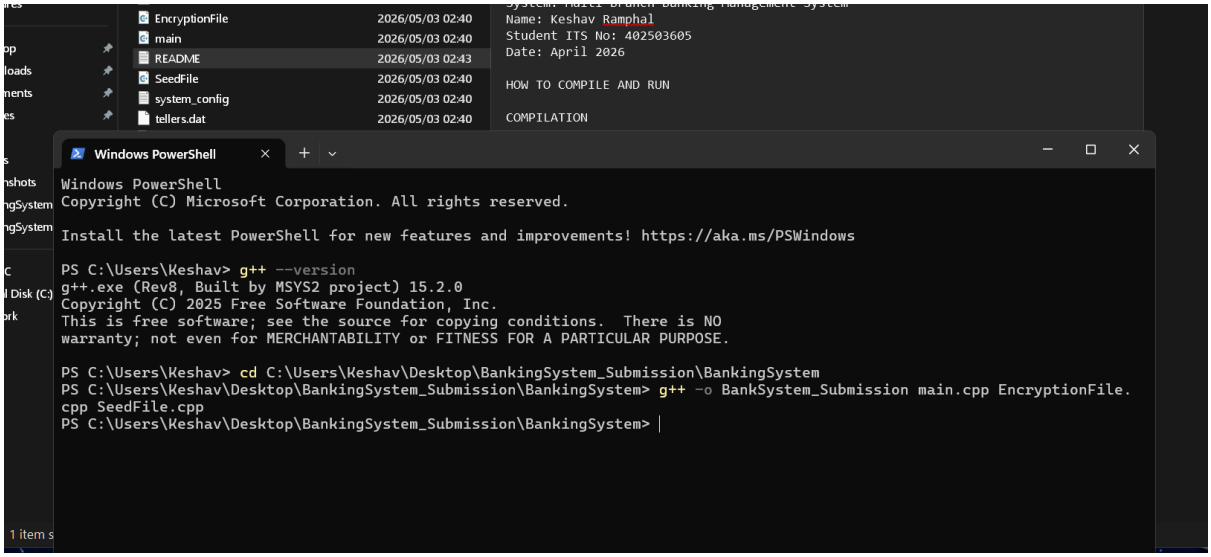

Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows
PS C:\Users\Keshav> g++ --version

then cd or go into your file and then after run it with ./BankSystem to create the set application at this point if it asked you to select a set application to open in then select non and go into the file that you extacted



You will find a README file with details like teller and customerr details to mess around with along with a application fthat will be builted by default


BankingSystem
Start backup
BankingSystem_Submission
BankingSystem
Search BankingSystem
New
Sort
View
Details
Home
Gallery
Keshav - Personal
Attachments
Documents
Pictures
Desktop
Downloads
Documents
Pictures
Music
Videos
Screenshots
BankingSystemFile
BankingSystem
This PC
Local Disk (C:)
Network
Name
Date modified
BankSystem 2026/05/03 02:40
BankSystem 2026/05/03 02:40
branches.dat 2026/05/03 02:40
customers.dat 2026/05/03 02:40
EncryptionFile 2026/05/03 02:40
main 2026/05/03 02:40
README 2026/05/03 02:43
SeedFile 2026/05/03 02:40
system_config 2026/05/03 02:40
tellers.dat 2026/05/03 02:40
transactions.dat 2026/05/03 02:40
CMakeLists
VERIFICATI
COMPILATI
READ
File
Edit
View
Assignment: PROGRAMMING 621 ASSIGNMENT
System: Multi-Branch Banking Management System
Name: Keshav Ramphal
Student ITS No: 402503605
Date: April 2026
HOW TO COMPILE AND RUN
COMPILATION
Unzip the folder
extract to desktop
go into the folder until you see set files like main.cpp
then copy the path to the folder
Open powershell
cd into the folder path you copied
g++ -o BankSystem_Submission main.cpp EncryptionFile.cpp SeedFile.cpp
./Banksystem
The system will:
1. Display the main menu
2. Automatically seed the database with default tellers and branches
3. Wait for user selection
BankSystem.h - Header file with structs, classes, and function declarations
main.cpp - Main program logic and menu systems
EncryptionFile.cpp - XOR encryption implementation
Ln 19, Col 1
70 of 1 987 charact
Plain text
100%
Unix (LF)
UTF-8


EncryptionFile 2026/05/03 02:40
main 2026/05/03 02:40
README 2026/05/03 02:43
SeedFile 2026/05/03 02:40
system_config 2026/05/03 02:40
tellers.dat 2026/05/03 02:40
Name: Keshav Ramphal
Student ITS No: 402503605
Date: April 2026
HOW TO COMPILE AND RUN
COMPILATION
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows
PS C:\Users\Keshav> g++ --version
g++.exe (Rev8, Built by MSYS2 project) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
PS C:\Users\Keshav> cd C:\Users\Keshav\Desktop\BankingSystem_Submission\BankingSystem
PS C:\Users\Keshav\Desktop\BankingSystem_Submission\BankingSystem> g++ -o BankSystem_Submission main.cpp EncryptionFile.
cpp SeedFile.cpp
PS C:\Users\Keshav\Desktop\BankingSystem_Submission\BankingSystem>

File Edit View H1

Assignment: PROGRAMMING 621 ASSIGNMENT
System: Multi-Branch Banking Management System
Name: Keshav Ramphal
Student ITS No: 402503605
Date: April 2026

HOW TO COMPILE AND RUN

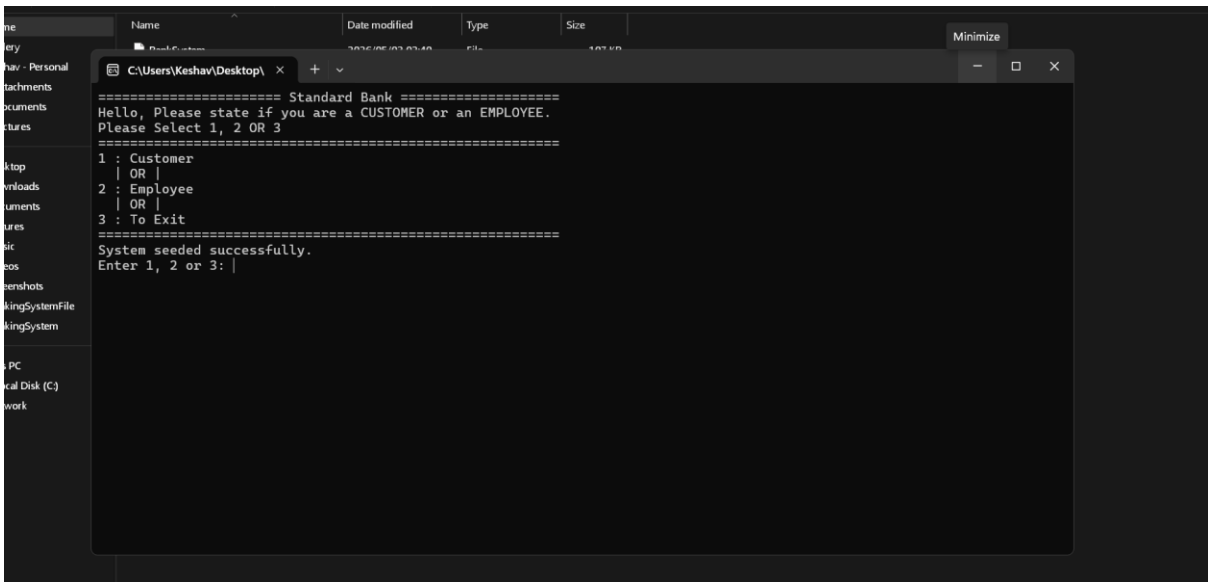
COMPILATION

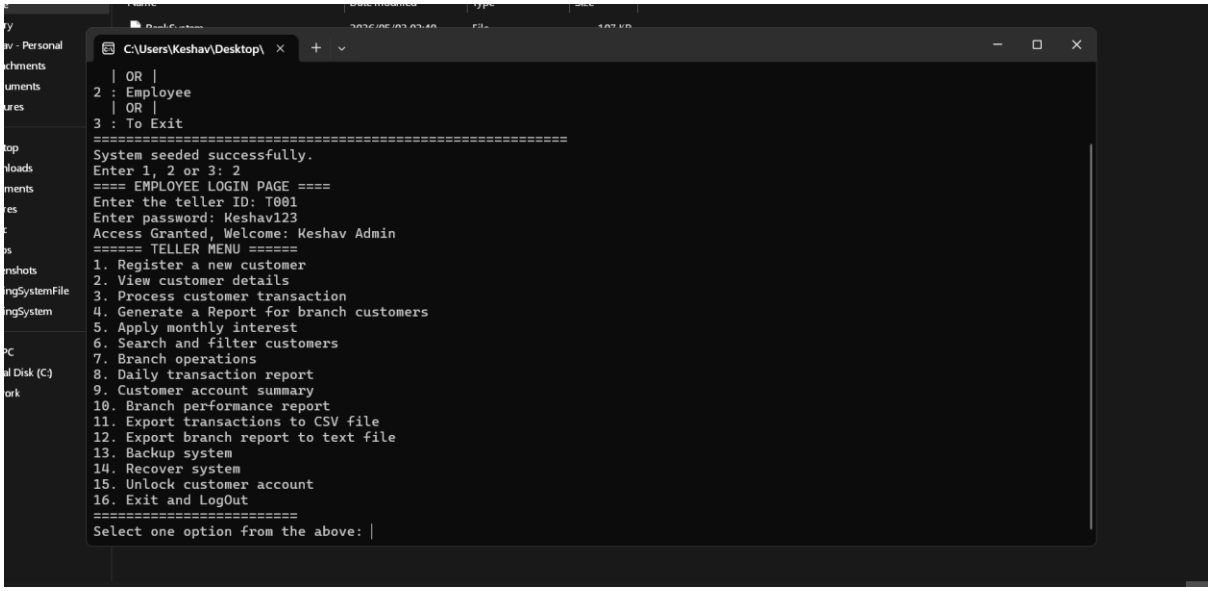
Unzip the folder
extract to desktop
go into the folder until you see set files like main.cpp
then copy the path to the folder
Open powershell

cd into the folder path you copied
 g++ -o BankSystem_Submission main.cpp EncryptionFile.cpp SeedFile.cpp
./Banksystem

The system will:
1. Display the main menu
2. Automatically seed the database with default tellers and branches
3. Wait for user selection

BankSystem.h - Header file with structs, classes, and function declarat
main.cpp - Main program logic and menu systems
EncryptionFile.cpp - XOR encryption implementation





The above images are of the program working and running.

I will submit the code that was used in the porcess

Firsr is the Header called BankSystem.h (w3schools, 2026).

```
#ifndef BANKINGSYSTEMFILE_BANKSYSTEM_H

#define BANKINGSYSTEMFILE_BANKSYSTEM_H


#include <fstream>

#include <ctime>

#include <cstring>

#include <iostream>

#include <string>
```

```
#include <cctype>
#include <cstddef>
using namespace std;

// File name constants

static const char CUSTOMERS_FILE[] = "customers.dat";
static const char TRANSACTIONS_FILE[] = "transactions.dat";
static const char TELLERS_FILE[] = "tellers.dat";
static const char BRANCHES_FILE[] = "branches.dat";
static const char CONFIG_FILE[] = "system_config.txt";
static const int MAX_LOGIN_ATTEMPTS = 3;

// Safe copy function to prevent buffer overflow
template <size_t N>
void safeCopy(char (&destination)[N], const string &source)
{
   strncpy(destination, source.c_str(), N - 1);
   destination[N - 1] = '\0';
}
// Teller struct - stores employee login info

struct Teller
{
```

```
    char tellerID[10];
    char tellerName[50];
    char tellerPassWord[50];
    char branchCode[10];
}; // Customer struct - stores customer account data

struct Customer
{
    char accountNumber[30];
    char accountName[50];
    char customerID[15];
    char contactNumber[12];
    char email[50];
    char physicalAddress[100];
    char dateOfBirth[11];
    char encryptedPin[50];
    double bankBalance;
    char branchCode[10];
    char accountType[20];
    int loginAttempts;
    bool isLocked;
};

// Branch struct
```

```
struct Branch
{
    char branchCode[10];
    char branchName[30];
    char city[40];
};

// Transaction struct

struct Transaction
{
    char accountNum[30];
    char targetAccount[50];
    char type[30];
    double amount;
    double balanceAfter;
    char branchCode[10];
    char tellerID[10];
    char date[20];
};

// Function prototypes

string xorDataEncryption(string data);
```

```
void seedSystem();
void logTransaction(
    const char *accNum,
    const char *type,
    double amount,
    double balanceAfter,
    const char *branchCode,
    const char *targetAccount = "",
    const char *tellerID = "SELF");

bool isValidSAID(string id);
bool isValid(const char *id);
bool isValidEmail(string email);
bool isValidContactNumber(string number);
bool isValidDOB(string dob);
string makeLower(string data);

// Base Account class for polymorphism

class Account
{
protected:
    double balance;
```

```
public:
    Account(double bal = 0.0)
    {
        balance = bal;
    }
    virtual double calculateInterest() = 0;
    virtual double minimumDeposit() = 0;
    virtual bool canWithdraw(double amount)
    {
        return amount > 0 && amount <= balance;
    }

    virtual ~Account() {}
}; // Savings Account - 5% interest

class SavingsAccount : public Account
{
public:
    SavingsAccount(double bal = 0.0) : Account(bal) {}
    double calculateInterest() override
    {
        return balance * 0.05;
    }
```

```
    double minimumDeposit() override
    {
        return 100.00;
    }
}; // Cheque Account - 2% interest
class ChequeAccount : public Account
{
public:
    ChequeAccount(double bal = 0.0) : Account(bal) {}

    double calculateInterest() override
    {
        return balance * 0.02;
    }

    double minimumDeposit() override
    {
        return 500.00;
    }
}; // Fixed Deposit Account - 8% interest, no withdrawals
class FixedDepositAccount : public Account
{
public:
    FixedDepositAccount(double bal = 0.0) : Account(bal) {}
```

```
    double calculateInterest() override
    {
        return balance * 0.08;
    }
    double minimumDeposit() override
    {
        return 1000.00;
    }
    bool canWithdraw(double amount) override
    {
        return false;
    }
}; // Student Account - 0% interest
class StudentAccount : public Account
{
public:
    StudentAccount(double bal = 0.0) : Account(bal) {}
    double calculateInterest() override
    {
        return 0.0;
    }
    double minimumDeposit() override
    {
        return 50.00;
```

```
    }
};

Account *createAccountObject(const char accountType[], double balance);

#endif
```

Second is the Encrytion file used for encytion of data (w3schools, 2026), (w3schools, 2026)

```
#include <iostream>
#include <string>
#include <sstream>
#include <iomanip>
#include "BankSystem.h"

using namespace std;

// XOR encryption - encrypts data and returns hex string

string xorDataEncryption(string data) {
    char keySet = 'K';
    stringstream output;
```

```
    for (int i = 0; i < (int)data.size(); i++) {
        int encryptedChar = ((unsigned char)data[i]) ^ keySet;
        output << hex << setw(2) << setfill('0') << encryptedChar;
        }
        return output.str();
    }
```

Third is the seed fle used to link thee files togther and save details etc. (w3schools, 2026)

```
#include <iostream>
#include <fstream>
#include <cstring>
#include "BankSystem.h"

using namespace std;
// Creates empty file if missing

void createFileIfMissing(const char fileName[]){
    ifstream testFile(fileName, ios::binary);
    if (!testFile)
    {
        ofstream createFile(fileName, ios::binary);
```

```
        createFile.close();
    }
    testFile.close();
}

// Seeds system with default tellers and branches
void seedSystem()
{
    createFileIfMissing(CUSTOMERS_FILE);
    createFileIfMissing(TRANSACTIONS_FILE);

    Teller tellers[3] = {};
    safeCopy(tellers[0].tellerID, "T001");
    safeCopy(tellers[0].tellerName, "Keshav Admin");
    safeCopy(tellers[0].tellerPassWord, xorDataEncryption("Keshav123"));
    safeCopy(tellers[0].branchCode, "Durban03");

    safeCopy(tellers[1].tellerID, "T002");
    safeCopy(tellers[1].tellerName, "Jozi Teller");
    safeCopy(tellers[1].tellerPassWord, xorDataEncryption("Jozi123"));
    safeCopy(tellers[1].branchCode, "Jozi06");

    safeCopy(tellers[2].tellerID, "T003");
    safeCopy(tellers[2].tellerName, "Cape Teller");
```

```
safeCopy(tellers[2].tellerPassWord, xorDataEncryption("Cape123"));
safeCopy(tellers[2].branchCode, "Cape01");

ofstream tellerfile(TELLERS_FILE, ios::binary);
tellerfile.write((char *)tellers, sizeof(tellers));
tellerfile.close();

Branch branches[3] = {};
safeCopy(branches[0].branchCode, "Durban03");
safeCopy(branches[0].branchName, "Durban North");
safeCopy(branches[0].city, "Durban");

safeCopy(branches[1].branchCode, "Jozi06");
safeCopy(branches[1].branchName, "Joburg CBD");
safeCopy(branches[1].city, "Johannesburg");

safeCopy(branches[2].branchCode, "Cape01");
safeCopy(branches[2].branchName, "Cape Town City");
safeCopy(branches[2].city, "Cape Town");

ofstream branchFile(BRANCHES_FILE, ios::binary);
branchFile.write((char *)branches, sizeof(branches));
branchFile.close();
```

```
    ifstream checkConfig(CONFIG_FILE);
    if (!checkConfig)
    {
        ofstream configFile(CONFIG_FILE);
        configFile << "BANK_NAME=Standard Bank Richfield" << endl;
        configFile << "VERSION=1.0" << endl;
        configFile << "MAINTENANCE_MODE=OFF" << endl;
        configFile << "MAX_LOGIN_ATTEMPTS=3" << endl;
        configFile << "DEFAULT_INTEREST_SAVINGS=0.05" << endl;
        configFile << "DEFAULT_INTEREST_CHEQUE=0.02" << endl;
        configFile << "DEFAULT_INTEREST_FIXED_DEPOSIT=0.08" << endl;
        configFile.close();
    }
    checkConfig.close();
    cout << "System seeded successfully." << endl;
}
```

Last is the main file where everything runs (Rozenberg, 2021), (w3schools, 2026), (tutorialspoints, 2026).

```
#include <cstring>
```

```
#include <fstream>
#include <iostream>
#include <string>
#include <ctime>
#include <cstdlib>
#include <limits>
#include <iomanip>
#include <cctype>

#include "BankSystem.h"

using namespace std;

int user_choice;
bool errorChecks = true;

// ========== INPUT VALIDATION HELPERS ==========

void clearInput() {
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

string getText(string message, int maxLength) {
```

```
    string input;
    while (true) {
        cout << message;
        getline(cin >> ws, input);
        if (input.length() == 0) {
            cout << "Input is empty, Please Insert" << endl;
        }
        else if ((int)input.length() > maxLength) {
            cout << "Input is too long, Maximum allowed: " << maxLength << endl;
        }
        else {
            return input;
        }
    }
}

int getIntInput(string message, int minValue, int maxValue) {
    int value;
    while (true) {
        cout << message;
        if (cin >> value && value >= minValue && value <= maxValue) {
            clearInput();
            return value;
        }
```

```
        cout << "Input is invalid, Please Insert a NUMBER from " << minValue << " to " << maxValue << endl;
        clearInput();
    }
}

double getDoubleInput(string message, double minValue) {
    double value;
    while (true) {
        cout << message;
        if (cin >> value && value >= minValue) {
            clearInput();
            return value;
        }
        cout << "Input is invalid, Amount must be at least R" << fixed << setprecision(2) << minValue << endl;
        clearInput();
    }
}

// ========== VALIDATION FUNCTIONS ==========

bool isDigitsOnly(string data) {
    if (data.length() == 0) {
        return false;
```

```
    }
    for (int i = 0; i < (int)data.length(); i++) {
        if (!isdigit((unsigned char)data[i])) {
            return false;
        }
    }
    return true;
}

bool isValidSAID(string id) {
    return id.length() == 13 && isDigitsOnly(id);
}

bool isValid(const char* id) {
    if (id == NULL) {
        return false;
    }
    return isValidSAID(string(id));
}

bool isValidEmail(string email) {
    size_t atPos = email.find("@");
    size_t dotPos = email.find(".", atPos);
```

```
    return atPos != string::npos && dotPos != string::npos && atPos > 0 && dotPos > atPos
+ 1 && dotPos < email.length() - 1;
}

bool isValidContactNumber(string number) {
    return number.length() == 10 && number[0] == '0' && isDigitsOnly(number);
}

bool isValidDOB(string dob) {
    if (dob.length() != 10) {
        return false;
    }
    if (dob[2] != '/' || dob[5] != '/') {
        return false;
    }
    string dayText = dob.substr(0, 2);
    string monthText = dob.substr(3, 2);
    string yearText = dob.substr(6, 4);
    if (!isDigitsOnly(dayText) || !isDigitsOnly(monthText) || !isDigitsOnly(yearText)) {
        return false;
    }
    int day = stoi(dayText);
    int month = stoi(monthText);
    int year = stoi(yearText);
```

```
    if (year < 1900 || year > 2100) {
        return false;
    }
    if (month < 1 || month > 12) {
        return false;
    }
    int daysInMonth[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    bool leapYear = (year % 400 == 0 || (year % 4 == 0 && year % 100 != 0));
    if (leapYear) {
        daysInMonth[1] = 29;
    }
    return day >= 1 && day <= daysInMonth[month - 1];
}

string makeLower(string data) {
    for (int i = 0; i < (int)data.length(); i++) {
        data[i] = tolower((unsigned char)data[i]);
    }
    return data;
}

// ========== ACCOUNT POLYMORPHISM ==========

Account* createAccountObject(const char accountType[], double balance) {
```

```
    if (strcmp(accountType, "Savings") == 0) {
        return new SavingsAccount(balance);
    }
    else if (strcmp(accountType, "Cheque") == 0) {
        return new ChequeAccount(balance);
    }
    else if (strcmp(accountType, "Fixed Deposit") == 0) {
        return new FixedDepositAccount(balance);
    }
    else if (strcmp(accountType, "Student") == 0) {
        return new StudentAccount(balance);
    }
    return NULL;
}

double getMinimumDeposit(const char accountType[]) {
    Account* acc = createAccountObject(accountType, 0.0);
    if (acc == NULL) {
        return 0.0;
    }
    double minDeposit = acc->minimumDeposit();
    delete acc;
    return minDeposit;
}
```

```
// ========== FILE OPERATIONS ==========

bool findCustomerByAccount(fstream& file, const char accountNum[], Customer&
tempCust, streampos& pos) {
   file.clear();
   file.seekg(0, ios::beg);
   while (file.read((char*)&tempCust, sizeof(Customer))) {
      pos = file.tellg();
      pos -= (streamoff)sizeof(Customer);
      if (strcmp(tempCust.accountNumber, accountNum) == 0) {
         return true;
      }
   }
   return false;
}

bool findCustomerReadOnly(const char accountNum[], Customer& tempCust) {
   ifstream inFile(CUSTOMERS_FILE, ios::binary);
   if (!inFile) {
      return false;
   }
   while (inFile.read((char*)&tempCust, sizeof(Customer))) {
      if (strcmp(tempCust.accountNumber, accountNum) == 0) {
```

```
            inFile.close();
            return true;
        }
    }
    inFile.close();
    return false;
}

bool writeCustomerRecord(fstream& file, streampos pos, Customer& tempCust) {
    file.clear();
    file.seekp(pos);
    file.write((char*)&tempCust, sizeof(Customer));
    file.flush();
    return !file.fail();
}

bool accountNumberExists(string accNum) {
    Customer tempCust;
    return findCustomerReadOnly(accNum.c_str(), tempCust);
}

bool pinExists(string encryptedPIN) {
    Customer tempCust;
    ifstream inFile(CUSTOMERS_FILE, ios::binary);
```

```
    if (!inFile) {
        return false;
    }
    while (inFile.read((char*)&tempCust, sizeof(Customer))) {
        if (strcmp(tempCust.encryptedPin, encryptedPIN.c_str()) == 0) {
            inFile.close();
            return true;
        }
    }
    inFile.close();
    return false;
}

string generateAccountNumber(char branch[]) {
    string accNum;
    for (int i = 0; i < 1000; i++) {
        int randomAccount = rand() % 90000 + 10000;
        accNum = "ACC-";
        accNum += branch;
        accNum += "-";
        accNum += to_string(randomAccount);
        if (!accountNumberExists(accNum)) {
            return accNum;
        }
```

```
    }
    return accNum;
}

string generatePIN() {
    string pin;
    for (int attempt = 0; attempt < 1000; attempt++) {
        pin = "";
        for (int j = 0; j < 5; j++) {
            pin += char('0' + rand() % 10);
        }
        string encrypted = xorDataEncryption(pin);
        if (!pinExists(encrypted)) {
            return pin;
        }
    }
    return pin;
}

void logTransaction(const char* accNum, const char* type, double amount, double
balanceAfter, const char* branchCode, const char* targetAccount, const char* tellerID) {
    Transaction t = {};
    safeCopy(t.accountNum, accNum);
    safeCopy(t.type, type);
```

```
    safeCopy(t.targetAccount, targetAccount);
    safeCopy(t.branchCode, branchCode);
    safeCopy(t.tellerID, tellerID);
    t.amount = amount;
    t.balanceAfter = balanceAfter;
    time_t now = time(0);
    strftime(t.date, 20, "%Y-%m-%d %H:%M", localtime(&now));
    ofstream outFile(TRANSACTIONS_FILE, ios::binary | ios::app);
    if (!outFile) {
        cout << "Unable to open file for writing." << endl;
        return;
    }
    outFile.write((char*)&t, sizeof(Transaction));
    outFile.close();
}

bool copyBinaryFile(const char sourceName[], const char destinationName[]) {
    ifstream src(sourceName, ios::binary);
    ofstream dst(destinationName, ios::binary);
    if (!src || !dst) {
        return false;
    }
    dst << src.rdbuf();
    src.close();
```

```
    dst.close();
    return true;
}

// ========== BACKUP AND RECOVERY ==========

void backupSystem() {
    bool success = true;
    success = copyBinaryFile(CUSTOMERS_FILE, "customers.bak") && success;
    success = copyBinaryFile(TRANSACTIONS_FILE, "transactions.bak") && success;
    success = copyBinaryFile(TELLERS_FILE, "tellers.bak") && success;
    success = copyBinaryFile(BRANCHES_FILE, "branches.bak") && success;
    success = copyBinaryFile(CONFIG_FILE, "system_config.bak") && success;
    if (success) {
        cout << "Backup system completed." << endl;
    }
    else {
        cout << "Backup system failed." << endl;
    }
}

void recoverSystem() {
    cout << "This will recover data from backup files." << endl;
    int choice = getIntInput("Enter 1 to continue or 2 to cancel: ", 1, 2);
```

```
    if (choice == 2) {
        cout << "Recovery system stopped." << endl;
        return;
    }
    bool success = true;
    success = copyBinaryFile("customers.bak", CUSTOMERS_FILE) && success;
    success = copyBinaryFile("transactions.bak", TRANSACTIONS_FILE) && success;
    if (success) {
        cout << "Recovery system completed." << endl;
    }
    else {
        cout << "Recovery system failed." << endl;
    }
}

// ========== EXPORT FUNCTIONS ==========

void exportTransactions() {
    ifstream inFile(TRANSACTIONS_FILE, ios::binary);
    if (!inFile) {
        cout << "Unable to open file for reading." << endl;
        return;
    }
    ofstream csvFile("TransactionHistory.csv");
```

```
    if (!csvFile) {
        cout << "Unable to open file for writing." << endl;
        return;
    }
    csvFile << "Account Number,Target Account,Type,Amount,Balance After,Branch Code,Teller ID,Date" << endl;
    Transaction t;
    while (inFile.read((char*)&t, sizeof(Transaction))) {
        csvFile << t.accountNum << ',' << t.targetAccount << ',' << t.type << ',' << fixed << setprecision(2) << t.amount << ',' << fixed << setprecision(2) << t.balanceAfter << ',' << t.branchCode << ',' << t.tellerID << ',' << t.date << endl;
    }
    inFile.close();
    csvFile.close();
    cout << "Transactions exported to TransactionHistory.csv" << endl;
}

void viewAccountStatement(const char accountNumber[]) {
    ifstream inFile(TRANSACTIONS_FILE, ios::binary);
    if (!inFile) {
        cout << "No data found." << endl;
        return;
    }
    Transaction t;
    bool found = false;
```

```
    cout << "======= ACCOUNT STATEMENT =======" << endl;
    cout << "Account Number: " << accountNumber << endl;
    cout << left << setw(18) << "Date" << setw(18) << "Type" << setw(20) << "Target" << setw(12) << "Amount" << setw(12) << "Balance" << setw(10) << "Teller" << endl;
    while (inFile.read((char*)&t, sizeof(Transaction))) {
        if (strcmp(t.accountNum, accountNumber) == 0) {
            found = true;
            cout << left << setw(18) << t.date << setw(18) << t.type << setw(20) << t.targetAccount << "R" << setw(11) << fixed << setprecision(2) << t.amount << "R" << setw(11) << fixed << setprecision(2) << t.balanceAfter << setw(10) << t.tellerID << endl;
        }
    }
    if (!found) {
        cout << "No data found." << endl;
    }
    inFile.close();
}

// ========== CUSTOMER AUTHENTICATION ==========

bool verifyCustomerPIN(Customer& currentCust, fstream& file, streampos pos, bool allowThreeTries) {
    while (true) {
        if (currentCust.isLocked) {
```

```
            cout << "Customer PIN is locked." << endl;
            return false;
        }
        string inputPIN = getText("Enter your PIN: ", 20);
        string encryptedPin = xorDataEncryption(inputPIN);
        if (inputPIN.length() == 5 && isDigitsOnly(inputPIN) &&
strcmp(currentCust.encryptedPin, encryptedPin.c_str()) == 0) {
            currentCust.loginAttempts = 0;
            currentCust.isLocked = false;
            writeCustomerRecord(file, pos, currentCust);
            return true;
        }
        else {
            currentCust.loginAttempts++;
            if (currentCust.loginAttempts >= MAX_LOGIN_ATTEMPTS) {
                currentCust.isLocked = true;
                writeCustomerRecord(file, pos, currentCust);
                cout << "Customer PIN is locked. Please visit your bank." << endl;
                return false;
            }
            else {
                writeCustomerRecord(file, pos, currentCust);
                cout << "Invalid PIN. Attempts left: " << MAX_LOGIN_ATTEMPTS -
currentCust.loginAttempts << endl;
            }
```

```
        }
        if (!allowThreeTries) {
            return false;
        }
    }
}

// ========== TRANSACTION FUNCTIONS ==========

bool depositMoney(fstream& file, Customer& currentCust, streampos pos, double amount, const char tellerID[]) {
    if (amount <= 0) {
        cout << "Invalid amount." << endl;
        return false;
    }
    currentCust.bankBalance += amount;
    if (!writeCustomerRecord(file, pos, currentCust)) {
        cout << "Error updating file" << endl;
        return false;
    }
    logTransaction(currentCust.accountNumber, "Deposit", amount, currentCust.bankBalance, currentCust.branchCode, "", tellerID);
    return true;
}
```

```
bool withdrawMoney(fstream& file, Customer& currentCust, streampos pos, double amount, const char tellerID[]) {
    if (amount <= 0) {
        cout << "Invalid amount." << endl;
        return false;
    }
    Account* acc = createAccountObject(currentCust.accountType, currentCust.bankBalance);
    if (acc == NULL) {
        cout << "Invalid account." << endl;
        return false;
    }
    bool allowed = acc->canWithdraw(amount);
    delete acc;
    if (!allowed) {
        cout << "Insufficient funds or withdrawal not allowed." << endl;
        return false;
    }
    currentCust.bankBalance -= amount;
    if (!writeCustomerRecord(file, pos, currentCust)) {
        cout << "Error updating file" << endl;
        return false;
    }
    logTransaction(currentCust.accountNumber, "Withdrawal", amount, currentCust.bankBalance, currentCust.branchCode, "", tellerID);
```

```
    return true;
}

bool transferMoney(fstream& file, Customer& sender, streampos senderPos, string
targetAccount, double amount, const char tellerID[]) {
    if (amount <= 0) {
        cout << "Invalid amount." << endl;
        return false;
    }
    if (targetAccount == sender.accountNumber) {
        cout << "Cannot send money to same account." << endl;
        return false;
    }
    Account* acc = createAccountObject(sender.accountType, sender.bankBalance);
    if (acc == NULL) {
        cout << "Invalid account." << endl;
        return false;
    }
    bool allowed = acc->canWithdraw(amount);
    delete acc;
    if (!allowed) {
        cout << "Insufficient funds or transfer not allowed." << endl;
        return false;
    }
```

```
    Customer receiver;
    streampos receiverPos;
    if (!findCustomerByAccount(file, targetAccount.c_str(), receiver, receiverPos)) {
        cout << "Account not found" << endl;
        return false;
    }
    sender.bankBalance -= amount;
    receiver.bankBalance += amount;
    writeCustomerRecord(file, senderPos, sender);
    writeCustomerRecord(file, receiverPos, receiver);
    logTransaction(sender.accountNumber, "Transfer Out", amount, sender.bankBalance,
sender.branchCode, receiver.accountNumber, tellerID);
    logTransaction(receiver.accountNumber, "Transfer In", amount, receiver.bankBalance,
receiver.branchCode, sender.accountNumber, tellerID);
    return true;
}

// ========== INTEREST APPLICATION ==========

void applyBranchInterest(char branchCode[]) {
    Customer tempCust;
    fstream file(CUSTOMERS_FILE, ios::binary | ios::in | ios::out);
    if (!file) {
        cout << "Error opening file, no customer." << endl;
        return;
```

```
    }
    cout << "Apply Monthly Interest for branch: " << branchCode << endl;
    int count = 0;
    while (file.read((char*)&tempCust, sizeof(Customer))) {
        streampos pos = file.tellg();
        pos -= (streamoff)sizeof(Customer);
        if (strcmp(tempCust.branchCode, branchCode) == 0) {
            Account* acc = createAccountObject(tempCust.accountType,
tempCust.bankBalance);
            if (acc != NULL) {
                double interest = acc->calculateInterest();
                delete acc;
                if (interest > 0) {
                    tempCust.bankBalance += interest;
                    file.clear();
                    file.seekp(pos);
                    file.write((char*)&tempCust, sizeof(Customer));
                    file.flush();
                    logTransaction(tempCust.accountNumber, "Monthly Interest", interest,
tempCust.bankBalance, tempCust.branchCode, "", "SYSTEM");
                    cout << "Applied R" << fixed << setprecision(2) << interest << " to account "
<< tempCust.accountNumber << endl;
                    count++;
                }
            }
```

```
        }
        file.clear();
        file.seekg(pos + (streamoff)sizeof(Customer), ios::beg);
    }
    file.close();
    cout << "Interest application complete. Updated " << count << " accounts." << endl;
}

// ========== DISPLAY FUNCTIONS ==========

void introduction_to_customer() {
    cout << "======================== Standard Bank ====================" << endl;
    cout << "Hello, Please state if you are a CUSTOMER or an EMPLOYEE." << endl;
    cout << "Please Select 1, 2 OR 3" << endl;
    cout << "============================================================" << endl;
    cout << "1 : Customer" << endl;
    cout << "  | OR |" << endl;
    cout << "2 : Employee" << endl;
    cout << "  | OR |" << endl;
    cout << "3 : To Exit" << endl;
    cout << "============================================================" << endl;
}
```

```
void generateBranchReport(char branch[]) {
    Customer tempCustomer;
    ifstream inFile(CUSTOMERS_FILE, ios::binary);
    if (!inFile) {
        cout << "Error opening file, no customer." << endl;
        return;
    }
    cout << "========== Customer Report for: " << branch << " ==========" << endl;
    cout << left << setw(22) << "Account Number" << setw(25) << "Name" << setw(18) << "Account Type" << "Balance" << endl;
    bool foundAny = false;
    while (inFile.read((char*)&tempCustomer, sizeof(Customer))) {
        if (strcmp(tempCustomer.branchCode, branch) == 0) {
            cout << left << setw(22) << tempCustomer.accountNumber << setw(25) << tempCustomer.accountName << setw(18) << tempCustomer.accountType << "R" << fixed << setprecision(2) << tempCustomer.bankBalance << endl;
            foundAny = true;
        }
    }
    inFile.close();
    if (!foundAny) {
        cout << "No Customers found for this branch." << endl;
    }
}
```

```
void displayCustomerDetails(Customer customer) {
    cout << "============ Customer Details ============" << endl;
    cout << "Account Number    : " << customer.accountNumber << endl;
    cout << "Account Name      : " << customer.accountName << endl;
    cout << "ID Number         : " << customer.customerID << endl;
    cout << "Contact Number    : " << customer.contactNumber << endl;
    cout << "Email             : " << customer.email << endl;
    cout << "Physical Address  : " << customer.physicalAddress << endl;
    cout << "Date of Birth     : " << customer.dateOfBirth << endl;
    cout << "Account Type      : " << customer.accountType << endl;
    cout << "Branch Code       : " << customer.branchCode << endl;
    cout << "Balance           : R" << fixed << setprecision(2) << customer.bankBalance <<
endl;
    cout << "Login Attempts    : " << customer.loginAttempts << endl;
    cout << "Locked            : " << (customer.isLocked ? "Yes" : "No") << endl;
    cout << "==========================================" << endl;
}

// ========== CUSTOMER REGISTRATION ==========

void registerCustomer(char branch[], const char tellerID[] = "TELLER") {
    Customer newCustomer = {};
    string plainPIN = "";
```

```
cout << "======== NEW CUSTOMER ========" << endl;
string name = getText("Enter Name: ", 49);
safeCopy(newCustomer.accountName, name);
string idNumber;
while (true) {
   idNumber = getText("Enter ID Number (13 digits): ", 14);
   if (isValidSAID(idNumber)) {
      safeCopy(newCustomer.customerID, idNumber);
      break;
   }
   else {
      cout << "Invalid ID Number!" << endl;
   }
}
string em;
while (true) {
   em = getText("Enter E-mail: ", 49);
   if (isValidEmail(em)) {
      safeCopy(newCustomer.email, em);
      break;
   }
   else {
      cout << "Invalid E-mail!" << endl;
   }
```

```
}
string contact;
while (true) {
    contact = getText("Enter Contact number (10 digits): ", 11);
    if (isValidContactNumber(contact)) {
        safeCopy(newCustomer.contactNumber, contact);
        break;
    }
    else {
        cout << "Invalid Contact Number!" << endl;
    }
}
string address = getText("Enter Address: ", 99);
safeCopy(newCustomer.physicalAddress, address);
string dob;
while (true) {
    dob = getText("Enter Date of Birth (DD/MM/YYYY): ", 10);
    if (isValidDOB(dob)) {
        safeCopy(newCustomer.dateOfBirth, dob);
        break;
    }
    else {
        cout << "Invalid DOB!" << endl;
    }
```

```
    }
    cout << "Select Account Type" << endl;
    cout << "1. Savings" << endl;
    cout << "2. Cheque" << endl;
    cout << "3. Fixed Deposit" << endl;
    cout << "4. Student" << endl;
    int typechoice = getIntInput("Enter your Choice: ", 1, 4);
    if (typechoice == 1) {
        safeCopy(newCustomer.accountType, "Savings");
    }
    else if (typechoice == 2) {
        safeCopy(newCustomer.accountType, "Cheque");
    }
    else if (typechoice == 3) {
        safeCopy(newCustomer.accountType, "Fixed Deposit");
    }
    else if (typechoice == 4) {
        safeCopy(newCustomer.accountType, "Student");
    }
    double minimumDeposit = getMinimumDeposit(newCustomer.accountType);
    cout << "Minimum Deposit for " << newCustomer.accountType << " account is R" <<
fixed << setprecision(2) << minimumDeposit << endl;
    double initialDeposit = getDoubleInput("Enter deposit amount: R", minimumDeposit);
    string accNum = generateAccountNumber(branch);
```

```
    safeCopy(newCustomer.accountNumber, accNum);
    plainPIN = generatePIN();
    string encrypted = xorDataEncryption(plainPIN);
    safeCopy(newCustomer.encryptedPin, encrypted);
    newCustomer.bankBalance = initialDeposit;
    safeCopy(newCustomer.branchCode, branch);
    newCustomer.loginAttempts = 0;
    newCustomer.isLocked = false;
    ofstream outFile(CUSTOMERS_FILE, ios::binary | ios::app);
    if (!outFile) {
        cout << "Error no file found" << endl;
        return;
    }
    outFile.write((char*)&newCustomer, sizeof(Customer));
    outFile.close();
    logTransaction(newCustomer.accountNumber, "Initial Deposit", initialDeposit,
newCustomer.bankBalance, newCustomer.branchCode, "", tellerID);
    cout << "Registration Successful" << endl;
    cout << "Account Number : " << newCustomer.accountNumber << endl;
    cout << "Temporary PIN  : " << plainPIN << endl;
    cout << "PLEASE DO NOT SHARE PIN WITH OTHERS" << endl;
}

// ========== PIN CHANGE ==========
```

```
void changePIN(fstream& file, Customer& currentCust, streampos pos) {
    string oldPIN = getText("Enter old PIN: ", 20);
    if (oldPIN.length() != 5 || !isDigitsOnly(oldPIN)) {
        cout << "Invalid old PIN!" << endl;
        return;
    }
    if (strcmp(currentCust.encryptedPin, xorDataEncryption(oldPIN).c_str()) != 0) {
        cout << "Old PIN is incorrect!" << endl;
        return;
    }
    string newPIN;
    while (true) {
        newPIN = getText("Enter new 5-digit PIN: ", 20);
        if (newPIN.length() == 5 && isDigitsOnly(newPIN)) {
            break;
        }
        cout << "PIN must be 5 digits" << endl;
    }
    string confirmPIN = getText("Confirm PIN: ", 20);
    if (newPIN != confirmPIN) {
        cout << "PIN confirmation does not match!" << endl;
        return;
    }
```

```
    safeCopy(currentCust.encryptedPin, xorDataEncryption(newPIN));
    currentCust.loginAttempts = 0;
    currentCust.isLocked = false;
    if (writeCustomerRecord(file, pos, currentCust)) {
        cout << "PIN changed successfully" << endl;
    }
    else {
        cout << "Error updating PIN" << endl;
    }
}

// ========== CUSTOMER LOGIN AND MENU ==========

void customer_files() {
    string inputAcc;
    Customer currentCust;
    bool loggedIn = false;
    streampos pos;
    cout << "=== Customer Login ===" << endl;
    inputAcc = getText("Enter Account Number: ", 29);
    fstream file(CUSTOMERS_FILE, ios::binary | ios::in | ios::out);
    if (!file) {
        cout << "Error no file found" << endl;
        return;
```

```
}
bool accountFound = findCustomerByAccount(file, inputAcc.c_str(), currentCust, pos);
if (!accountFound) {
    cout << "Invalid Account Number!" << endl;
    file.close();
    return;
}
if (currentCust.isLocked) {
    cout << "Account Locked!" << endl;
    file.close();
    return;
}
if (verifyCustomerPIN(currentCust, file, pos, true)) {
    loggedIn = true;
    cout << "Welcome: " << currentCust.accountName << endl;
}
if (loggedIn) {
    int custChoice;
    bool custSession = true;
    while (custSession) {
        cout << "================ CUSTOMER MENU ================" << endl;
        cout << "1. Check Balance" << endl;
        cout << "2. Deposit" << endl;
        cout << "3. Withdrawal" << endl;
```

```
        cout << "4. Transfer" << endl;
        cout << "5. View Account Statement" << endl;
        cout << "6. Change PIN" << endl;
        cout << "7. Log out" << endl;
        custChoice = getIntInput("Enter your Choice: ", 1, 7);
        if (custChoice == 1) {
            cout << "Balance: R" << fixed << setprecision(2) << currentCust.bankBalance <<
endl;
        }
        else if (custChoice == 2) {
            double amount = getDoubleInput("Enter deposit amount: R", 0.01);
            if (depositMoney(file, currentCust, pos, amount, "SELF")) {
                cout << "Deposit Successful. Balance: R" << fixed << setprecision(2) <<
currentCust.bankBalance << endl;
            }
        }
        else if (custChoice == 3) {
            double amount = getDoubleInput("Enter withdrawal amount: R", 0.01);
            if (withdrawMoney(file, currentCust, pos, amount, "SELF")) {
                cout << "Withdrawal Successful. Balance: R" << fixed << setprecision(2) <<
currentCust.bankBalance << endl;
            }
        }
        else if (custChoice == 4) {
            string targetAccount = getText("Enter target account number: ", 29);
```

```
            double targetamount = getDoubleInput("Enter transfer amount: R", 0.01);
            if (transferMoney(file, currentCust, pos, targetAccount, targetamount, "SELF"))
{
                cout << "Transfer Successful. Balance: R" << fixed << setprecision(2) <<
currentCust.bankBalance << endl;
            }
        }
        else if (custChoice == 5) {
            viewAccountStatement(currentCust.accountNumber);
        }
        else if (custChoice == 6) {
            changePIN(file, currentCust, pos);
        }
        else if (custChoice == 7) {
            cout << "Logging out" << endl;
            custSession = false;
        }
    }
  }
  file.close();
}

// ========== TELLER FUNCTIONS ==========

void viewCustomerDetailsForTeller(char branch[]) {
```

```
    string acc = getText("Enter Account Number: ", 29);
    Customer tempCustomer;
    if (!findCustomerReadOnly(acc.c_str(), tempCustomer)) {
        cout << "No customer found" << endl;
        return;
    }
    if (strcmp(tempCustomer.branchCode, branch) != 0) {
        cout << "Access denied. Customer not in your branch." << endl;
        return;
    }
    displayCustomerDetails(tempCustomer);
}

void tellerProcessTransaction(Teller teller) {
    string acc = getText("Enter Account Number: ", 29);
    fstream file(CUSTOMERS_FILE, ios::binary | ios::in | ios::out);
    if (!file) {
        cout << "Error no file found" << endl;
        return;
    }
    Customer tempCustomer;
    streampos pos;
    if (!findCustomerByAccount(file, acc.c_str(), tempCustomer, pos)) {
        cout << "No customer found" << endl;
```

```
    file.close();
    return;
}
if (strcmp(tempCustomer.branchCode, teller.branchCode) != 0) {
    cout << "Access denied. Customer not in your branch." << endl;
    file.close();
    return;
}
if (tempCustomer.isLocked) {
    cout << "Account is locked!" << endl;
    file.close();
    return;
}
cout << "Customer PIN verification required." << endl;
if (!verifyCustomerPIN(tempCustomer, file, pos, true)) {
    file.close();
    return;
}
cout << "1. Cash Deposit" << endl;
cout << "2. Cash Withdrawal" << endl;
cout << "3. Fund Transfer" << endl;
cout << "4. Account Statement" << endl;
int choice = getIntInput("Enter your Choice: ", 1, 4);
if (choice == 1) {
```

```
        double amount = getDoubleInput("Enter Deposit amount: R", 0.01);
        if (depositMoney(file, tempCustomer, pos, amount, teller.tellerID)) {
            cout << "Deposit Successful." << endl;
        }
    }
    else if (choice == 2) {
        double amount = getDoubleInput("Enter Withdrawal amount: R", 0.01);
        if (withdrawMoney(file, tempCustomer, pos, amount, teller.tellerID)) {
            cout << "Withdrawal Successful." << endl;
        }
    }
    else if (choice == 3) {
        string target = getText("Enter target Account Number: ", 29);
        double amount = getDoubleInput("Enter Transfer Amount: R", 0.01);
        if (transferMoney(file, tempCustomer, pos, target, amount, teller.tellerID)) {
            cout << "Transfer Successful." << endl;
        }
    }
    else if (choice == 4) {
        viewAccountStatement(tempCustomer.accountNumber);
    }
    file.close();
}
```

```
// ========== SEARCH AND FILTER ==========

void searchCustomer(char branch[]) {
    cout << "1. Search by Account Number" << endl;
    cout << "2. Search by Name" << endl;
    cout << "3. Search by ID Number" << endl;
    cout << "4. Filter by Account Type" << endl;
    int choice = getIntInput("Enter choice: ", 1, 4);
    string searchTerm = getText("Enter search value: ", 49);
    string loweredSearch = makeLower(searchTerm);
    ifstream inFile(CUSTOMERS_FILE, ios::binary);
    if (!inFile) {
        cout << "No customer data found." << endl;
        return;
    }
    Customer tempCustomer;
    bool found = false;
    while (inFile.read((char*)&tempCustomer, sizeof(Customer))) {
        if (strcmp(tempCustomer.branchCode, branch) != 0) {
            continue;
        }
        bool match = false;
        if (choice == 1 && searchTerm == tempCustomer.accountNumber) {
            match = true;
```

```
        }
        else if (choice == 2 &&
makeLower(tempCustomer.accountName).find(loweredSearch) != string::npos) {
            match = true;
        }
        else if (choice == 3 && searchTerm == tempCustomer.customerID) {
            match = true;
        }
        else if (choice == 4 &&
makeLower(tempCustomer.accountType).find(loweredSearch) != string::npos) {
            match = true;
        }
        if (match) {
            displayCustomerDetails(tempCustomer);
            found = true;
        }
    }
    inFile.close();
    if (!found) {
        cout << "No matching customers found." << endl;
    }
}

// ========== BRANCH OPERATIONS ==========
```

```
void viewAllBranches() {
    ifstream inFile(BRANCHES_FILE, ios::binary);
    if (!inFile) {
        cout << "No branches found." << endl;
        return;
    }
    Branch branch;
    cout << "================ ALL BRANCHES ================" << endl;
    while (inFile.read((char*)&branch, sizeof(Branch))) {
        cout << "Branch Code: " << branch.branchCode << endl;
        cout << "Branch Name: " << branch.branchName << endl;
        cout << "City       : " << branch.city << endl;
        cout << "-----------------------------------------------" << endl;
    }
    inFile.close();
}

void viewBranchDetails() {
    string code = getText("Enter branch code: ", 9);
    ifstream inFile(BRANCHES_FILE, ios::binary);
    if (!inFile) {
        cout << "No branches found." << endl;
        return;
    }
```

```
    Branch branch;
    bool found = false;
    while (inFile.read((char*)&branch, sizeof(Branch))) {
        if (code == branch.branchCode) {
            cout << "Branch Code: " << branch.branchCode << endl;
            cout << "Branch Name: " << branch.branchName << endl;
            cout << "City       : " << branch.city << endl;
            found = true;
            break;
        }
    }
    inFile.close();
    if (!found) {
        cout << "Branch not found." << endl;
    }
}

void branchPerformanceReport() {
    ifstream branchFile(BRANCHES_FILE, ios::binary);
    if (!branchFile) {
        cout << "No branch data found." << endl;
        return;
    }
    Branch branch;
```

```
    cout << "================ BRANCH PERFORMANCE REPORT
================" << endl;
    while (branchFile.read((char*)&branch, sizeof(Branch))) {
        ifstream customerFile(CUSTOMERS_FILE, ios::binary);
        Customer tempCustomer;
        int customerCount = 0;
        double totalBalance = 0.0;
        while (customerFile.read((char*)&tempCustomer, sizeof(Customer))) {
            if (strcmp(tempCustomer.branchCode, branch.branchCode) == 0) {
                customerCount++;
                totalBalance += tempCustomer.bankBalance;
            }
        }
        customerFile.close();
        cout << "Branch: " << branch.branchCode << " - " << branch.branchName << endl;
        cout << "Customers: " << customerCount << endl;
        cout << "Total Balance: R" << fixed << setprecision(2) << totalBalance << endl;
        cout << "------------------------------------------------------------" << endl;
    }
    branchFile.close();
}

void branchOperationsMenu() {
    bool branchSession = true;
```

```
    while (branchSession) {
        cout << "========= BRANCH OPERATIONS =========" << endl;
        cout << "1. View All Branches" << endl;
        cout << "2. View Branch Details" << endl;
        cout << "3. Inter-Branch Comparison" << endl;
        cout << "4. Back" << endl;
        int choice = getIntInput("Enter choice: ", 1, 4);
        if (choice == 1) {
            viewAllBranches();
        }
        else if (choice == 2) {
            viewBranchDetails();
        }
        else if (choice == 3) {
            branchPerformanceReport();
        }
        else if (choice == 4) {
            branchSession = false;
        }
    }
}

// ========== REPORTS ==========
```

```
void dailyTransactionReport(char branch[]) {
    ifstream inFile(TRANSACTIONS_FILE, ios::binary);
    if (!inFile) {
        cout << "No transaction data found." << endl;
        return;
    }
    char today[11];
    time_t now = time(0);
    strftime(today, 11, "%Y-%m-%d", localtime(&now));
    Transaction t;
    bool found = false;
    int count = 0;
    double total = 0.0;
    cout << "================ DAILY TRANSACTION REPORT ================"
<< endl;
    cout << "Date: " << today << endl;
    while (inFile.read((char*)&t, sizeof(Transaction))) {
        if (strncmp(t.date, today, 10) == 0 && strcmp(t.branchCode, branch) == 0) {
            cout << t.date << " | " << t.accountNum << " | " << t.type << " | R" << fixed <<
setprecision(2) << t.amount << endl;
            found = true;
            count++;
            total += t.amount;
        }
    }
```

```
    inFile.close();
    if (!found) {
        cout << "No transactions for today." << endl;
    }
    cout << "Total Transactions: " << count << endl;
    cout << "Total Volume: R" << fixed << setprecision(2) << total << endl;
}

void customerAccountSummary(char branch[]) {
    ifstream inFile(CUSTOMERS_FILE, ios::binary);
    if (!inFile) {
        cout << "No customer data found." << endl;
        return;
    }
    Customer tempCustomer;
    int totalCustomers = 0;
    int savings = 0;
    int cheque = 0;
    int fixedDeposit = 0;
    int student = 0;
    double totalBalance = 0.0;
    while (inFile.read((char*)&tempCustomer, sizeof(Customer))) {
        if (strcmp(tempCustomer.branchCode, branch) == 0) {
            totalCustomers++;
```

```
            totalBalance += tempCustomer.bankBalance;
            if (strcmp(tempCustomer.accountType, "Savings") == 0) {
                savings++;
            }
            else if (strcmp(tempCustomer.accountType, "Cheque") == 0) {
                cheque++;
            }
            else if (strcmp(tempCustomer.accountType, "Fixed Deposit") == 0) {
                fixedDeposit++;
            }
            else if (strcmp(tempCustomer.accountType, "Student") == 0) {
                student++;
            }
        }
    }
    inFile.close();
    cout << "================ CUSTOMER ACCOUNT SUMMARY ================"
<< endl;
    cout << "Branch: " << branch << endl;
    cout << "Total Customers: " << totalCustomers << endl;
    cout << "Savings Accounts: " << savings << endl;
    cout << "Cheque Accounts: " << cheque << endl;
    cout << "Fixed Deposit Accounts: " << fixedDeposit << endl;
    cout << "Student Accounts: " << student << endl;
```

```
    cout << "Total Balance: R" << fixed << setprecision(2) << totalBalance << endl;
}

void exportBranchReportText(char branch[]) {
    string fileName = "BranchReport_";
    fileName += branch;
    fileName += ".txt";
    ofstream outFile(fileName.c_str());
    if (!outFile) {
        cout << "Could not create text report." << endl;
        return;
    }
    ifstream inFile(CUSTOMERS_FILE, ios::binary);
    if (!inFile) {
        cout << "No customer data found." << endl;
        return;
    }
    Customer tempCustomer;
    outFile << "Customer Report for Branch: " << branch << endl;
    outFile << "Account Number | Name | Account Type | Balance" << endl;
    while (inFile.read((char*)&tempCustomer, sizeof(Customer))) {
        if (strcmp(tempCustomer.branchCode, branch) == 0) {
            outFile << tempCustomer.accountNumber << " | " << tempCustomer.accountName
<< " | " << tempCustomer.accountType << " | R" << fixed << setprecision(2) <<
tempCustomer.bankBalance << endl;
```

```
        }
    }
    inFile.close();
    outFile.close();
    cout << "Branch report exported to " << fileName << endl;
}

// ========== UNLOCK CUSTOMER ==========

void unlockCustomer(char branch[]) {
    string acc = getText("Enter account number to unlock: ", 29);
    fstream file(CUSTOMERS_FILE, ios::binary | ios::in | ios::out);
    if (!file) {
        cout << "Customer file not found." << endl;
        return;
    }
    Customer tempCustomer;
    streampos pos;
    if (!findCustomerByAccount(file, acc.c_str(), tempCustomer, pos)) {
        cout << "Customer not found." << endl;
        file.close();
        return;
    }
    if (strcmp(tempCustomer.branchCode, branch) != 0) {
```

```
        cout << "Access denied. Customer not in your branch." << endl;
        file.close();
        return;
    }
    tempCustomer.loginAttempts = 0;
    tempCustomer.isLocked = false;
    writeCustomerRecord(file, pos, tempCustomer);
    file.close();
    cout << "Customer account unlocked successfully." << endl;
}

// ========== EMPLOYEE LOGIN AND MENU ==========

void employee_files() {
    string inputID;
    string inputPassword;
    bool found = false;
    Teller tempTeller;
    cout << "==== EMPLOYEE LOGIN PAGE ====" << endl;
    inputID = getText("Enter the teller ID: ", 9);
    inputPassword = getText("Enter password: ", 24);
    string encryptedPassword = xorDataEncryption(inputPassword);
    ifstream inFile(TELLERS_FILE, ios::binary);
    if (!inFile) {
```

```
        cout << "Error no teller file found" << endl;
        return;
    }
    while (inFile.read((char*)&tempTeller, sizeof(tempTeller))) {
        if (strcmp(tempTeller.tellerID, inputID.c_str()) == 0 &&
strcmp(tempTeller.tellerPassWord, encryptedPassword.c_str()) == 0) {
            found = true;
            break;
        }
    }
    inFile.close();
    if (found) {
        cout << "Access Granted, Welcome: " << tempTeller.tellerName << endl;
    }
    else {
        cout << "Error, no teller found or invalid password." << endl;
        return;
    }
    int tellerChoice;
    bool inSession = true;
    while (inSession) {
        cout << "====== TELLER MENU ======" << endl;
        cout << "1. Register a new customer" << endl;
        cout << "2. View customer details" << endl;
```

```
cout << "3. Process customer transaction" << endl;
cout << "4. Generate a Report for branch customers" << endl;
cout << "5. Apply monthly interest" << endl;
cout << "6. Search and filter customers" << endl;
cout << "7. Branch operations" << endl;
cout << "8. Daily transaction report" << endl;
cout << "9. Customer account summary" << endl;
cout << "10. Branch performance report" << endl;
cout << "11. Export transactions to CSV file" << endl;
cout << "12. Export branch report to text file" << endl;
cout << "13. Backup system" << endl;
cout << "14. Recover system" << endl;
cout << "15. Unlock customer account" << endl;
cout << "16. Exit and LogOut" << endl;
cout << "========================" << endl;
tellerChoice = getIntInput("Select one option from the above: ", 1, 16);
if (tellerChoice == 1) {
   registerCustomer(tempTeller.branchCode, tempTeller.tellerID);
}
else if (tellerChoice == 2) {
   viewCustomerDetailsForTeller(tempTeller.branchCode);
}
else if (tellerChoice == 3) {
   tellerProcessTransaction(tempTeller);
```

```
}
else if (tellerChoice == 4) {
    generateBranchReport(tempTeller.branchCode);
}
else if (tellerChoice == 5) {
    applyBranchInterest(tempTeller.branchCode);
}
else if (tellerChoice == 6) {
    searchCustomer(tempTeller.branchCode);
}
else if (tellerChoice == 7) {
    branchOperationsMenu();
}
else if (tellerChoice == 8) {
    dailyTransactionReport(tempTeller.branchCode);
}
else if (tellerChoice == 9) {
    customerAccountSummary(tempTeller.branchCode);
}
else if (tellerChoice == 10) {
    branchPerformanceReport();
}
else if (tellerChoice == 11) {
    exportTransactions();
```

```
        }
        else if (tellerChoice == 12) {
            exportBranchReportText(tempTeller.branchCode);
        }
        else if (tellerChoice == 13) {
            backupSystem();
        }
        else if (tellerChoice == 14) {
            recoverSystem();
        }
        else if (tellerChoice == 15) {
            unlockCustomer(tempTeller.branchCode);
        }
        else if (tellerChoice == 16) {
            cout << "Logging out" << endl;
            inSession = false;
        }
    }
}

// ========== MAIN MENU ==========

void function_selection() {
    while (errorChecks) {
```

```
        user_choice = getIntInput("Enter 1, 2 or 3: ", 1, 3);
        if (user_choice == 1) {
            customer_files();
        }
        else if (user_choice == 2) {
            employee_files();
        }
        else if (user_choice == 3) {
            cout << "GoodBye and have a good day." << endl;
            errorChecks = false;
        }
    }
}

// ========== MAIN ==========

int main() {
    srand((unsigned int)time(0));
    introduction_to_customer();
    seedSystem();
    function_selection();
    return 0;
}
```

Conclusion

To build a set system it takes a lot of time, sweat and legit hours in order to code no stop in order to get a system that works at its maximum

References

Rozenberg, Z. (2021, September 14). *Incredibuild*. Retrieved from Incredibuild: https://www.incredibuild.com/blog/cstdlib-in-c-explained

tutorialspoints. (2026, May 03). *tutorialspoints*. Retrieved from tutorialspoints: https://www.tutorialspoint.com/cpp_A_library/limits.htm

w3schools. (2026, April 15). *w3schools*. Retrieved from w3schools: https://www.w3schools.com/programming/prog_programming.php

w3schools. (2026, April). *w3schools*. Retrieved from w3schools: https://www.geeksforgeeks.org/cpp/file-handling-c-classes/

w3schools. (2026, April 20). *w3schools*. Retrieved from w3schools: https://www.geeksforgeeks.org/cpp/ctype-hcctype-library-in-c-c-with-examples/

w3schools. (2026, April 30). *w3schools*. Retrieved from w3schools: https://www.geeksforgeeks.org/cpp/iomanip-in-cpp/

w3schools. (2026, April). *w3schools*. Retrieved from w3schools: https://www.geeksforgeeks.org/cpp/how-to-seed-a-random-number-generator-in-cpp/

w3schools. (2026, April 02). *w3schools*. Retrieved from w3schools: https://www.w3schools.com/cpp/cpp_input_validation.asp